

PULSE · ENGINEERING PROPOSAL

Play Fingerprint

Recognising plays we have seen before, and proving it. Shazam-style retrieval for football events.

Item	Detail
Status	Proposal v0.1, for discussion. Nothing here is built into the product yet.
Date	Tue 6 Oct 2026
Audience	Pulse engineers (pipeline and agents)
Tier	Should. Not on the golden path. Parked automatically if the gate on Tue 13 Oct fails.
Reads with	AGENTS.md (rules), PHASES.md (dates and cut order), contracts/ (schemas)

Naming note. Shazam is a third-party trademark. The hackathon rules for the demo video ban third-party trademarks, so the name stays in this internal document only. In the UI, video and README we call the feature **Play Fingerprint** and the viewer chip **Seen before**.

1. The idea on one page

Shazam can name a song from a few noisy seconds of audio. It does this by turning the audio into points, hashing pairs of points into compact addresses, and looking those addresses up in an inverted index. The right song is the one where many hits line up at the same time offset.

We can do the same thing with football. A **play** (a possession chain or a detected moment) is an ordered list of events. We turn each event into a token, hash pairs of tokens, and store them in an index of earlier plays. When a new play unfolds, we look it up and answer three questions: have we seen this before, where, and how many times?

Why it is worth building

- **A demo hook that is easy to remember.** Click a moment, and Pulse shows the earlier plays it matches, side by side on the mini pitch.
- **It makes claims about the past checkable.** Language models invent history ("third time this half"). Here every history claim is a lookup result with event IDs, and the Verifier compares the wording to it.
- **It fits the architecture as it stands.** Deterministic Go code produces facts, agents narrate them, the Verifier checks them. The index is one more fact source. No new agent is needed.

What it is not

- **Not an AI model.** The index is deterministic code. The AI part is the Narrator describing the result and the Verifier checking that description.
- **Not a speed trick.** Our library will hold a few thousand plays, and a linear scan would be fast enough. Do not claim performance in the pitch. The value is the capability and the verifiability.
- **Not audio.** No audio fingerprinting is involved, and no copyrighted music is touched.

Decision requested

Approve Play Fingerprint as a **Should** item with a go/no-go gate at the Tue 13 Oct stand-up, a build window of Tue 13 to Sat 17 Oct, and a hard stop on the evening of Sat 17 Oct. Section 7 has the tasks, dates and the proposed cut-order position.

Safety rule. Nothing in this proposal touches the golden path, the Verifier-with-fallback, the Evidence Trail or deployment. If the gate fails we lose nothing and no existing work depends on it.

2. What we borrow from Shazam, and what we leave

The Shazam article is mostly about surviving noisy microphones. Our input is already clean, discrete events, so half of it does not apply. The half that does is the combinatorial hashing and the voting.

Shazam idea	Pulse equivalent	Verdict
Spectrogram, FFT, peak picking (the "constellation map")	Not needed. Events are already discrete. We tokenise them instead.	Drop
Noise tolerance	A play with dropped or jittered events should still match. Tested explicitly (section 8).	Keep as a goal
Anchor point and a target zone of 5 points	Anchor is one event token. Target zone is the next five tokens in the play.	Keep
32-bit address: anchor frequency, point frequency, time delta	32-bit address: anchor token, target token, time-gap bucket.	Keep, adapted
Inverted index: address to [anchor time, song id]	Address to [anchor position, play id].	Keep, adapted
Discard songs with too few shared target zones	Discard plays with fewer than N coherent pairs.	Keep
Time coherency: histogram of time offsets	Histogram of position offsets within the play (see below).	Keep, adapted
Sharding the database for scale	Not needed at our library size.	Drop
Query by humming, similarity search	Query by sketch: draw a route on the pitch, find similar plays.	Won't (for now)

Why we adapt the time axis

Shazam assumes a recording plays at the same speed as the original, so a 10 second clip lines up at one clean offset. Football plays do not keep that promise: the same pressing trap might unfold over 6 seconds or 11. So we do two things differently.

- The time gap inside each address is a **coarse, log-scaled bucket**, not milliseconds. A slightly faster or slower version of the same play still produces the same addresses.
- The coherence vote uses the **position of the event within the play** (event number 4, event number 5) instead of absolute time. Slow and fast versions of one play agree on the position offset even when their clocks do not.

This is a design hypothesis, not a proven result. Section 8 tests it with sped-up and slowed-down copies of the same play.

3. Why it fits "facts first"

Pulse already has one rule above all others: agents narrate only from a fact pack, and a Verifier checks every claim against event IDs. Play Fingerprint extends that rule to history. The flow is:

1. The engine segments the match into plays and indexes every completed one.
2. When a moment fires, the engine looks up the live play and attaches the result to the fact pack as `similar_plays`.
3. The Explainer and Narrator may mention history only by referencing those entries.
4. The Verifier rejects any history claim that has no entry behind it, or whose numbers or scope do not match.
5. Two rejections fall back to the template, as for every other claim. The cue gate is unchanged.

What the Verifier accepts and rejects

Narrator sentence	Result	Why
"Kestrel have built an attack this way twice before in our match library, most recently in match 12."	Accept	Two <code>SimilarPlay</code> entries exist. Count and match ID agree. Scope phrase present.
"Kestrel have done this five times before."	Reject	<code>earlier_matches</code> is 2. Code <code>HISTORY_COUNT_MISMATCH</code> .
"This is the best pressing trap of the season."	Reject	No fact supports "best" or "season". Code <code>HISTORY_UNSUPPORTED</code> .
"Teams have used this pattern in football for years."	Reject	Our library is synthetic. Claims must be scoped to it. Code <code>HISTORY_SCOPE</code> .

Scope wording is a hard rule. Our library is synthetic, so "seen before" only ever means "seen before in our match library". The Narrator prompt and the Verifier both enforce that phrase. This keeps the demo honest and it stops the feature from claiming real-world discoveries.

4. How it works

4.1 What counts as a play

A **play** is an ordered list of event IDs. We index two kinds, using the same code for both:

- **Chain.** A possession chain, from a `POSSESSION_CHANGE` (or kick-off) to the next `POSSESSION_CHANGE` or `WHISTLE`.
- **Moment.** The event window that the moment detector already produces for each detected moment (the engine already attaches event IDs to moments).

4.2 Tokenise each event

Each event becomes a 10-bit **token**: 4 bits for the event kind and 6 bits for a pitch zone.

- **Kinds that become tokens:** `PASS`, `CARRY` and `SHOT` by the team in possession, `PRESSURE` by the opposing team, and `POSSESSION_CHANGE` as a marker. `WHISTLE` ends a play and is not tokenised. The other event types in the contract can be added later by widening the kind field.
- **Zones:** an 8 by 8 grid over the pitch gives 64 zones. Coordinates come from the contract's `PitchPoint` (confirm the unit and origin in `common.ts`).

- **Attack-direction normalisation:** flip x when the possessing team is attacking the other way, so every play is read left to right. Without this, the same move in the second half never matches the first.
- **Mirroring:** left-to-right mirror matching (a left-wing attack matching a right-wing one) is a config flag, off by default. Decide in section 12.

4.3 Pair the tokens and pack an address

For each token (the anchor) we pair it with the next five tokens (its target zone). A play of n tokens produces about 5n pairs. Each pair becomes a 32-bit address.

Field	Bits	Notes
Anchor token	10 (bits 22 to 31)	Event kind plus zone of the anchor.
Target token	10 (bits 12 to 21)	Event kind plus zone of the target.
Time-gap bucket	4 (bits 8 to 11)	Log-scaled gap between the two events. Seven buckets used (below). Nine values spare.
Reserved	8 (bits 0 to 7)	Left at zero. Room for a layout version or a play-kind flag.

Bucket	Gap between events	Reason
0	under 0.25 s	Near-simultaneous (pressure and pass release).
1 to 2	0.25 to 1 s	One-touch and quick combinations.
3 to 4	1 to 4 s	Normal build-up.
5	4 to 8 s	Slow circulation.
6	8 s and over	Long gap. Rare inside one play.

These layouts are a starting point. All the numbers (grid size, bucket edges, target zone size) live in one config struct so the sweep in section 8 can tune them without code changes.

4.4 Build the index

The index maps each address to a list of couples, where a couple is {position of the anchor in its play, play ID}. It is held in memory, built at engine start-up from the library, and inserted in sorted order so the result is the same every run. We log a checksum of the built index so CI can assert determinism.

4.5 Look up a play

1. For every pair in the query play, fetch the couples stored at its address.
2. Each couple votes for (play ID, offset), where offset = candidate anchor position minus query anchor position.
3. For each candidate play, take the offset with the most votes. That count is its **coherent pairs**.
4. Similarity = coherent pairs divided by the query's pair count, capped at 1.
5. Discard candidates below **minCoherent** and **minSimilarity**. Sort by similarity, then by play ID so the order is deterministic. Return the top K.

Because similarity is measured against the **query's** pairs, a play that is still being built can match the opening of a complete earlier one. This gives us the "this is building like the late siege" behaviour for free. Early in a play there are few pairs, so require a minimum (start at 8) before looking up.

4.6 Worked example

Query play, attack-normalised: a pass from zone 18, a pass from zone 27, a carry from zone 36, then a shot from zone 52. Timestamps are 0 s, 1.2 s, 2.9 s and 5.1 s. The first three pairs, anchored on the first pass, are:

Pair	Anchor token	Target token	Gap	Bucket
1	PASS, zone 18	PASS, zone 27	1.2 s	3
2	PASS, zone 18	CARRY, zone 36	2.9 s	4
3	PASS, zone 18	SHOT, zone 52	5.1 s	5

An earlier play that contains the same pass, pass, carry, shot pattern, even if it starts three events later or runs 20 percent slower, produces the same addresses. All of its pairs vote for one offset, so it wins by a wide margin. An unrelated play shares a few addresses by chance, but its votes scatter across offsets and fall under the threshold.

5. Contract and agent changes

The agent messages (**FactPack**, **NarrationDraft**, **VerificationResult**) are still on the Phase 0 list. This proposal adds one optional block to **FactPack**. It is additive, so it does not break contract v1 and can ship as v1.1.

5.1 The SimilarPlay type

Field	Type	Rule
play_id	Id	The matched play.
match_id	Id	The match that play came from.
kind	CHAIN MOMENT	What was indexed.
similarity	number, 0 to 1	Must equal <code>coherent_pairs / query_pairs</code> within 1e-6.
coherent_pairs	integer, 1 or more	Must not exceed <code>query_pairs</code> .
query_pairs	integer	Pair count of the query play.
evidence	Id[], 1 or more	Event IDs of the matched play. The Evidence Trail replays these.
label	string, optional	Moment type if known, for example <code>PRESSING_TRAP</code> .

The fact pack also carries two numbers: **library_size** (matches in the index) and **earlier_matches** (distinct plays above threshold). **earlier_matches** must be at least the length of **similar_plays**, because the list is truncated to the top K.

As with every other rule in the contract, the cross-field checks (similarity arithmetic, non-empty evidence) cannot be expressed in JSON Schema alone, so they live in code in each language. Ship one valid fixture and at least three invalid ones: similarity that does not match the arithmetic, empty evidence, and **coherent_pairs** greater than **query_pairs**.

5.2 Engine (pipeline)

- New package for the tokeniser, pairing, index and lookup. Proposed location: `app/engine/fingerprint/` (confirm against the repo map).
- MCP tool `find_similar_plays(play_id | event_ids, k)` returns the same struct, so agents and evals can re-run a lookup.
- Fact pack builder fills `similar_plays`, `library_size` and `earlier_matches` whenever a moment fires.

5.3 Agents

- **Explainer:** mentions history only when `similar_plays` is non-empty. Otherwise stays silent about the past.
- **Narrator:** a history claim must cite the `SimilarPlay` it comes from, and must use the scope phrase "in our match library".
- **Verifier:** four new checks, each with a rejection code so failures are searchable in logs.
 - `HISTORY_UNSUPPORTED`: a recurrence or ranking claim with no entry behind it (including "best", "first ever", "every time").
 - `HISTORY_COUNT_MISMATCH`: a number that does not equal `earlier_matches`.
 - `HISTORY_SCOPE`: missing scope phrase, or a claim about "the season", "the league" or "football".
 - `HISTORY_MATCH_UNKNOWN`: names a match or play that is not in `similar_plays`.
- **Evals:** add at least 10 cases: 4 supported, 3 count mismatch, 2 unsupported superlative, 1 scope. The existing bar stays: Verifier false-accept rate is zero on invented-claim cases.

5.4 Viewer

- A **Seen before xN** chip on cues that carry `similar_plays`.
- Clicking it opens the Evidence Trail with the current play and the best match side by side on the mini pitch (reuse `pitch.ts`), with the event IDs listed.
- Fallback if the pitch work slips: a plain list of matched plays with their match IDs and evidence links.

6. The library

A fingerprint index is only as good as what is in it. Our library is generated, so we control it, and we should be open about that.

- **Contents:** the three scenario matches plus about 30 seeded filler matches from the Go generator. That should yield a few thousand plays. Measure the real number once the generator runs.
- **Planted patterns:** each demo scenario must have deliberately recurring plays in earlier matches, so the demo is guaranteed to find a match. This is fine for synthetic data, and the README must say so plainly: "synthetic library with seeded recurring patterns". We never present it as real-world discovery.
- **Variety:** the filler matches need enough variation that not every play looks alike. Add pattern templates and noise knobs in `data/scenarios/` and keep the seeds in one file.
- **Determinism:** same seed gives the same library gives the same index checksum. CI asserts this.
- **Leakage:** when we evaluate a scenario, its own play is excluded from the library so it cannot match itself.

Check the generator first. If the generator cannot produce recurring patterns with natural variation, the whole feature is hollow. Task FP1 below is deliberately first and deliberately small so we learn this on day one of the build window.

7. Plan

7.1 Gate: Tue 13 Oct stand-up

We start the build only if all three are true. If any fails, Play Fingerprint is parked without discussion.

- The three scenarios run end to end with real agents.
- Verifier false-accept rate is zero on the current eval set.
- Evidence Trail is started (a branch exists that replays events on the mini pitch). The side-by-side view depends on it.

7.2 Tasks

ID	Task	Owner	Est.	Done when
FP1	Generator: recurring pattern templates and variety knobs for the filler matches	P	0.5 d	30 seeded matches generate and the data-quality checks still pass
FP2	Tokeniser, attack normalisation, pairing, address packing	P	0.5 d	Unit tests pass, address round-trip proven
FP3	Index build and lookup with coherence voting and deterministic ordering	P	0.5 d	Self-match and partial-play tests pass
FP4	Golden, robustness and false-positive tests; config sweep	P	0.5 d	Pass bars in section 8 met
FP5	<code>SimilarPlay</code> schema, fixtures, Go and Python models	S	0.25 d	Shared fixtures accepted and rejected correctly in both languages
FP6	Fact pack fields and MCP tool <code>find_similar_plays</code>	P	0.5 d	Agent service can call the tool and read the fields
FP7	Prompts, four Verifier checks, 10 new eval cases	A	1 d	Zero false accepts on the new cases
FP8	Viewer: Seen before chip and side-by-side pitch (or list fallback)	P	1 d	Works on the Azure URL in judge mode
FP9	README wording, judge instructions, 20-second demo beat	A	0.25 d	Copy reviewed, no trademarks

That is about 3.25 days of pipeline time, 1.25 days of agents time and 0.25 days shared.

7.3 Calendar

When	What
Tue 13 Oct	Gate. FP1 starts first thing.
Wed 14 to Thu 15 Oct	FP2, FP3, FP4 (P). FP5 (S). Thu is integration day.
Fri 16 to Sat 17 Oct	FP6 and FP8 (P). FP7 and FP9 (A).
Sat 17 Oct, evening	Hard stop. If the FP4 pass bars are not green, cut the feature and keep the work on a branch.
Sun 18 Oct	Phase 3 exit: the full demo path runs on the public URL.

The honest risk. Phase 3 is already the heaviest phase: Evidence Trail, Player Spotlight, CI/CD, Swahili and Speech all land in the same days. This feature is attractive, but it is the first thing to lose if the Evidence Trail is not finished by Thu 15 Oct.

7.4 Proposed change to the cut order

Current: Newcomer and Loyalist, then Recap agent, then Event Hubs, then Speech audio, then Player Spotlight, then Swahili.

Proposed: Newcomer and Loyalist, then Recap agent, then **Play Fingerprint**, then Event Hubs, then Speech audio, then Player Spotlight, then Swahili. This is a proposal for the team to confirm, not a decision. The Never-cut list is unchanged.

8. Testing and quality

A fingerprint feature is easy to demo and easy to get subtly wrong, so the tests carry more weight than usual. All pass bars below are targets to measure, not promises.

Test	What it proves	Pass bar
Address round-trip	Packing and unpacking an address loses nothing. Fields do not overlap.	Property test, 10,000 random triples
Determinism	Same seed gives the same library and the same index checksum.	Identical checksum across runs and in CI
Self-match	Every play in the library finds itself first.	Top-1 for 100 percent
Partial-play match	A half-built play matches the opening of its complete version.	Top-1 for 100 percent at 9 or more events
Dropped events	Randomly removing 20 percent of events still finds the play (Shazam's noise tolerance, translated).	Top-1 for at least 90 percent
Time jitter and tempo	Timestamps shifted by up to 300 ms, and plays sped up or slowed down by 25 percent, still match.	Top-1 for at least 90 percent
Direction invariance	The same play attacking left or right gives the same result.	Identical top-1 and similarity
False positives	Unrelated plays return nothing above threshold.	Zero on 200 or more unrelated queries
Exclusion	A scenario never matches itself in the demo library.	Asserted in the golden test
Contract	Valid SimilarPlay accepted, invalid rejected, in both languages.	All shared fixtures agree
Verifier evals	The four history checks accept supported claims and reject invented ones.	Zero false accepts

Early evidence

As a sanity check we wrote a Go sketch of the core (Appendix A) and a test file covering four of these cases. On 500 randomly generated synthetic plays, every play found itself first, partial plays found their complete version, dropping 20 percent of events still gave top-1 for all 500, and 200 unrelated plays produced no false matches.

Read that result carefully. Random plays are much easier to tell apart than real football, where many possessions share the same common patterns (back-passes, switches, throw-ins). The sketch shows the mechanics work. It does not show the parameters are right. The tuning sweep in FP4 has to run on generator output, and the false-positive test matters most there.

Tooling

- Table-driven Go tests with golden files for the scenario lookups. Keep the lint and format setup you already use (golangci-lint for Go, ESLint and Prettier for the TypeScript overlay).
- Keep every tunable value in one config struct and every test reading it, so a parameter change shows up in one diff.
- Log the rejection code for every Verifier failure so we can report rates in the evals write-up.

9. Risks

Risk	Early signal	Response
Tokens too coarse: everything matches everything	False-positive test fails, or similarity is high on unrelated plays	Raise the minimum, add the zone or bucket detail, shrink the target zone
Tokens too fine: nothing matches	Dropped-event and jitter tests fail	Coarsen the grid or the bucket edges, widen the target zone
Generator cannot produce natural recurring patterns	FP1 overruns half a day	Hand-author the recurring plays for the three scenarios only; skip the filler variety
Feature competes with the Evidence Trail and Phase 3	Evidence Trail not done by Thu 15 Oct	Stop. Cut per section 7.4.
Claims overreach ("in football", "this season")	A scoped phrase missing in review of outputs	Verifier HISTORY_SCOPE check. Treat a miss as a release blocker.
Judges think the data is real	Questions in the Q&A, or comments	State "synthetic library with seeded patterns" in the README, the video and the UI footer
Trademark in public material	The word Shazam appears in video, UI or README	Search the repo and script before freeze. Use Play Fingerprint and Seen before only.
Non-deterministic output	Golden tests flaky	Sorted insertion, deterministic tie-break, no map-order dependence (the sketch already does this)

10. The 20-second demo beat

1. A late-siege moment banner appears with a Verified badge on the Narrator line.
2. The presenter clicks **Seen before x2** on the cue.
3. The Evidence Trail opens. The mini pitch shows the current build-up and the match 12 build-up side by side, with event IDs listed.
4. The Narrator line reads: "Kestrel have built an attack this way twice before in our match library, most recently in match 12." The Verifier tick shows.

Pitch wording, trademark-safe: "Audio recognition taught phones to name a song from a few noisy seconds. Pulse recognises a play from a few seconds of football, and every claim it makes about the past is a lookup, not a guess." Check the exact wording against the rules before we publish it.

11. Related: Pulse Daily (the podcast question)

The podcast idea works as an **output**, and it does not need audio fingerprinting. We generate the audio ourselves, so we already know its timeline, and Azure Speech word boundaries give word-level timing.

- Reuse the Recap agent and Speech: a 3 to 4 minute two-voice recap after full time, downloadable as an MP3.
- Chapters link to each moment's Evidence Trail, so a listener can jump from a spoken moment to the events behind it.
- Play Fingerprint adds at most one "seen before" line to the script, subject to the same Verifier checks.
- No new agent and no live radio. Match Radio stays in Won't. Pulse Daily is Should, and it sits behind Speech audio and the Recap agent in the cut order, so it is cut first.

12. Open decisions

#	Decision	Recommendation	Who decides
1	Run the 13 Oct gate and the build window	Yes, with the hard stop on Sat 17 Oct	Both
2	Position in the cut order	Third, after Recap (section 7.4)	Both
3	Index both chains and moments, or moments only	Both. Moments only is the cut-down version if FP2 slips.	P
4	Mirror matching (left wing equals right wing)	Off by default. Test with the flag on, decide with data.	P
5	Verifier re-runs the lookup, or only checks the fact pack	Fact pack check in production. Re-run through MCP only in evals.	A
6	Where the <code>SimilarPlay</code> schema lives and when	<code>contracts/</code> , with the agent messages in Phase 0 so FactPack is defined once	S
7	Pulse Daily as a Should	Yes, only after Speech works	A

Items to confirm in the real repo: the engine package path, the unit and origin of `PitchPoint`, the list of event kinds in the contract, and the exact commands for tests and lint. AGENTS.md currently marks the commands as "confirm".

Appendix A. Go sketch of the core

A working sketch of tokens, addresses, pairing, the index and lookup. It compiles, passes **go vet**, and passes the four tests described in section 8. It is a starting point for FP2 and FP3, not production code: it has no error handling for oversized plays (anchor positions are **uint16**), and the tokeniser from raw events is not included.

```
package fingerprint

import (
    "math"
    "sort"
)

// EventKind is the 4-bit event class used in a token.
type EventKind uint8

// Token is one "note" in a play: what happened and where (attack-normalised).
// Layout: kind (4 bits) | zone (6 bits).
type Token uint16

func NewToken(kind EventKind, zone uint8) Token {
    return Token(uint16(kind&0xF)<<6 | uint16(zone&0x3F))
}

// DtBucket is a coarse, log-scaled time gap between two events (0..15).
type DtBucket uint8

// Address packs anchor, target and time gap into 32 bits.
// Layout: anchor (10) | target (10) | dt (4) | reserved (8).
type Address uint32

func NewAddress(anchor, target Token, dt DtBucket) Address {
    return Address(uint32(anchor&0xFF)<<22 | uint32(target&0xFF)<<12 | uint32(dt&0xF)<<8)
}

// Note is a tokenised event with its time inside a play.
type Note struct {
    Token Token
    AtMs  int64
}

// Pair is one anchor/target combination from a play.
type Pair struct {
    Addr      Address
    AnchorIdx int
}

const targetZoneSize = 5

// Pairs pairs each note (the anchor) with the next five notes (its target zone).
func Pairs(notes []Note, bucket func(gapMs int64) DtBucket) []Pair {
    var out []Pair
    for i := range notes {
        for j := i + 1; j < len(notes) && j <= i+targetZoneSize; j++ {
            out = append(out, Pair{
                Addr:      NewAddress(notes[i].Token, notes[j].Token, bucket(notes[j].AtMs-notes[i].AtMs)),
                AnchorIdx: i,
            })
        }
    }
    return out
}

// Couple is what the index stores per address.
type Couple struct {
    AnchorIdx uint16
    PlayID    uint32
}

type Index struct {
```

```

    postings map[Address][]Couple
}

func NewIndex() *Index { return &Index{postings: make(map[Address][]Couple)} }

func (ix *Index) Add(playID uint32, pairs []Pair) {
    for _, p := range pairs {
        ix.postings[p.Addr] = append(ix.postings[p.Addr],
            Couple{AnchorIdx: uint16(p.AnchorIdx), PlayID: playID})
    }
}

type Match struct {
    PlayID      uint32
    CoherentPairs int
    QueryPairs   int
    Similarity   float64
}

type vote struct {
    playID uint32
    offset int
}

// Lookup returns the plays most similar to the query, best first.
// Ties are broken by PlayID so the output is deterministic.
func (ix *Index) Lookup(query []Pair, minCoherent int, minSimilarity float64, k int) []Match {
    if len(query) == 0 {
        return nil
    }
    votes := make(map[vote]int)
    for _, p := range query {
        for _, c := range ix.postings[p.Addr] {
            votes[vote{c.PlayID, int(c.AnchorIdx) - p.AnchorIdx}]++
        }
    }
    best := make(map[uint32]int) // playID -> votes at its best offset
    for v, n := range votes {
        if n > best[v.playID] {
            best[v.playID] = n
        }
    }
    var out []Match
    for id, n := range best {
        sim := math.Min(1, float64(n)/float64(len(query)))
        if n < minCoherent || sim < minSimilarity {
            continue
        }
        out = append(out, Match{PlayID: id, CoherentPairs: n, QueryPairs: len(query), Similarity: sim})
    }
    sort.Slice(out, func(a, b int) bool {
        if out[a].Similarity != out[b].Similarity {
            return out[a].Similarity > out[b].Similarity
        }
        return out[a].PlayID < out[b].PlayID
    })
    if len(out) > k {
        out = out[:k]
    }
    return out
}

```

Appendix B. Source note

The technique comes from the Coding Geek article "How does Shazam work". The parts we use are:

- Anchor points paired with a target zone of five points.
- A 32-bit address packed from the anchor, the point and the time delta.
- An inverted index where lookup per address is constant time.
- Filtering by the number of shared target zones, then confirming with time coherency.

We do not use the audio processing (FFT, Hamming windows, downsampling, band-wise peak picking), and the article's reported speed-up from this design does not carry over because our library is small. Nothing in this proposal uses audio recordings, lyrics or any third-party content.